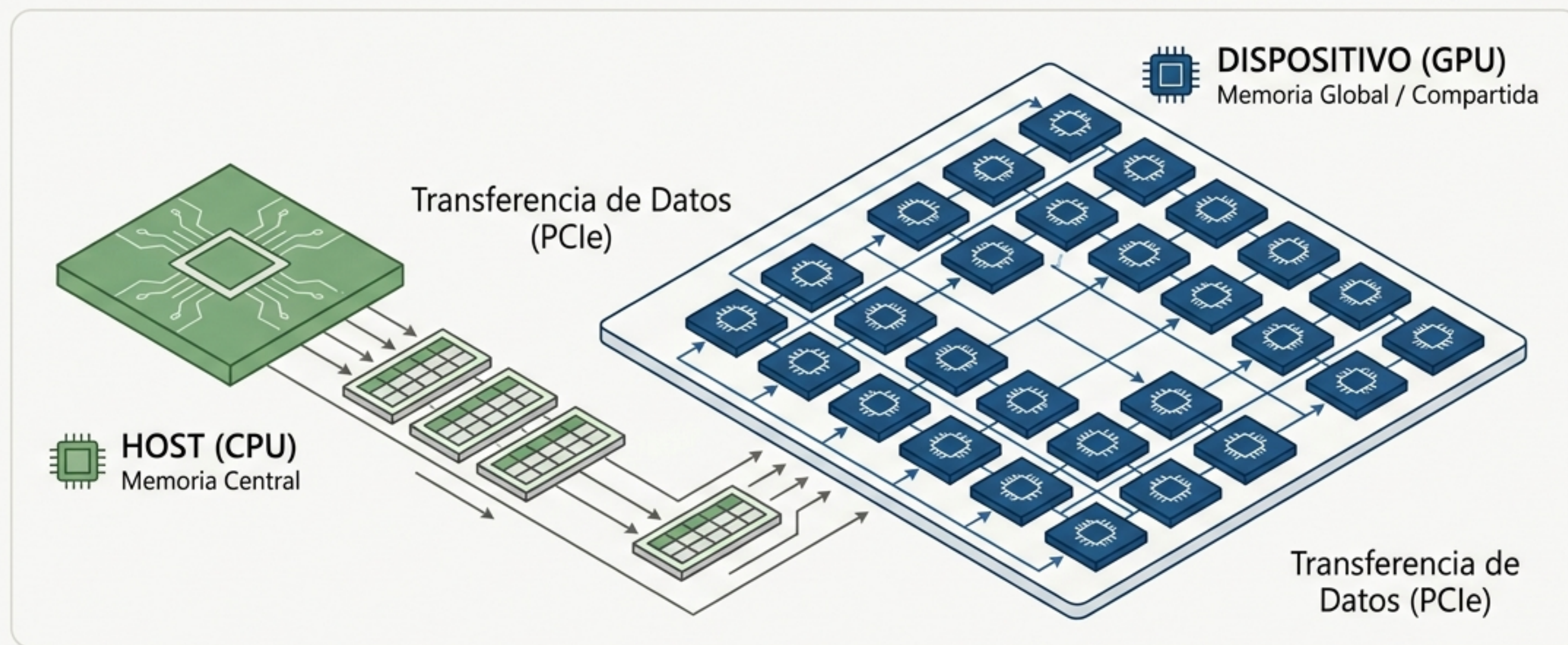


Programación en CUDA: De Cero a Paralelismo 2D

Guía técnica paso a paso: Entorno, Memoria y Computación Matricial



Configuración del Entorno (Ground Zero)

Antes de la ejecución paralela, verificamos la infraestructura. nvidia-smi revela la capacidad de cómputo y el estado de la memoria VRAM de la GPU. Para transformar el código en instrucciones, utilizamos el compilador propietario de NVIDIA, nvcc.

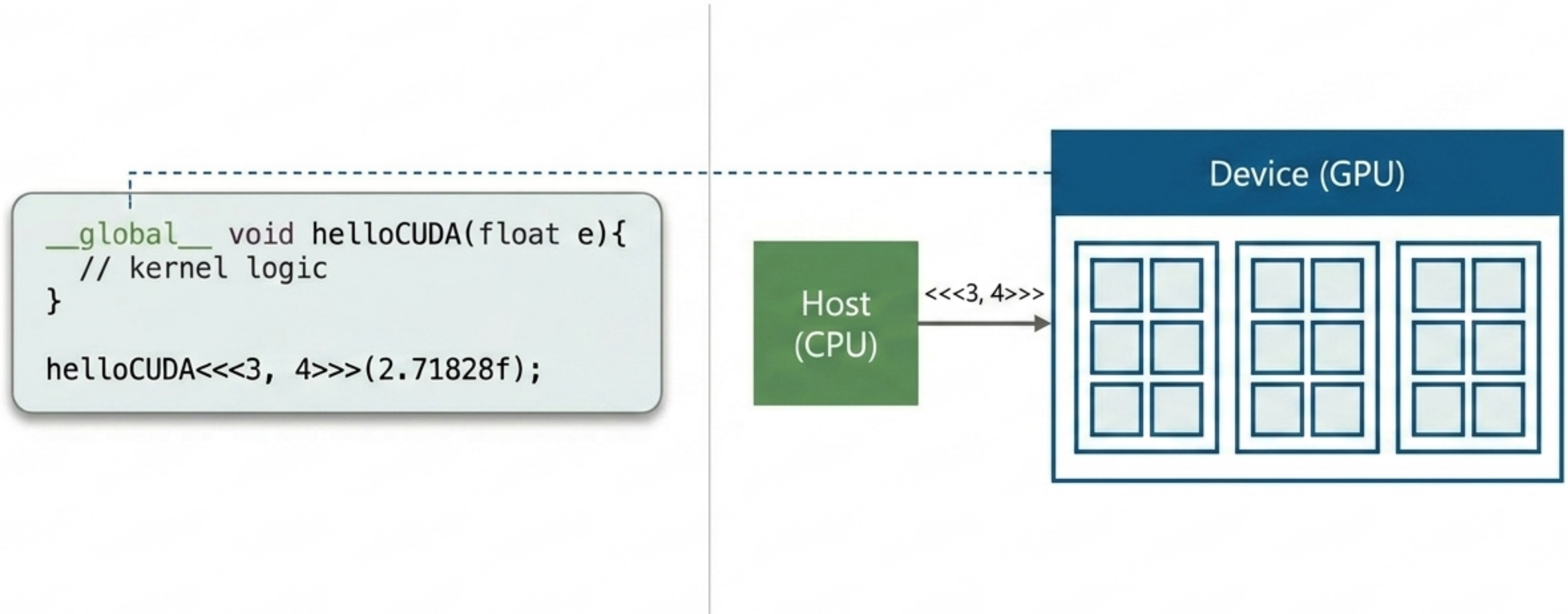
Mon May 4 21:42:17 2026

NVIDIA-SMI 580.82.07			Driver Version: 580.82.07			CUDA Version: 13.0		
GPU	Name		Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr. ECC	
Fan	Temp	Perf	Pwr/Usage/Cap		Memory-Usage	GPU-Util	Compute M.	
							MIG M.	
0	Tesla T4		Off	00000000:00:04.0	Off		0	
N/A	38C	P8	9W / 70W	0MiB / 15360MiB		0%	Default	N/A
Processes:								
GPU	GI	CI	PID	Type	Process name	GPU Memory		
	ID	ID				Usage		
No running processes found								

```
nvcc hello.cu -o hello
./hello
```

Fundamentos: Hello World y la Sintaxis del Kernel

El identificador `__global__` indica que el Kernel será ejecutado en el Device (GPU) pero invocado desde el Host (CPU). La configuración de ejecución `<<<bloques, hilos>>>` define la topología del lanzamiento.



Identidad del Hilo: Coordenadas Espaciales en la GPU

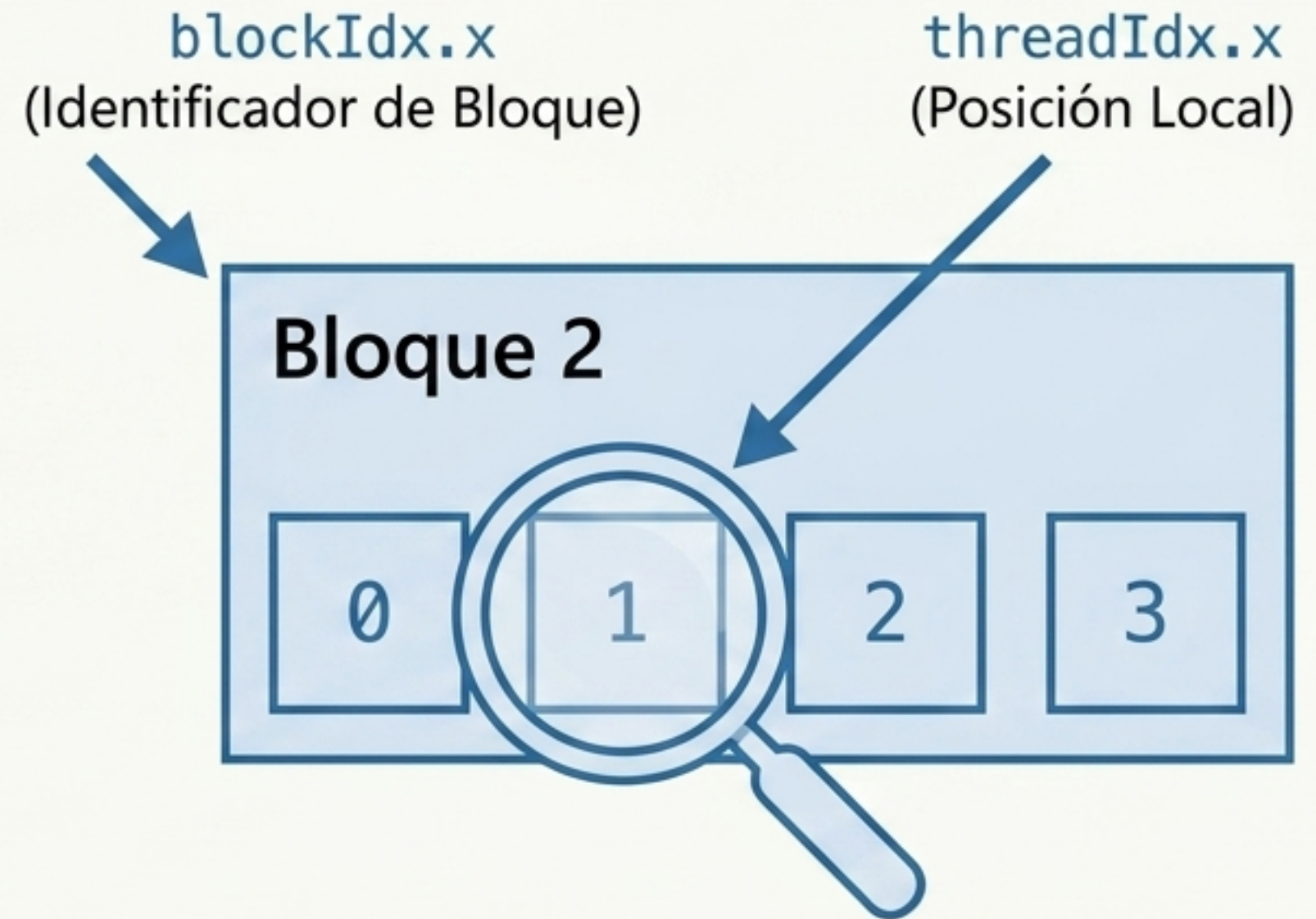
Miles de hilos ejecutan la misma instrucción simultáneamente. Cada hilo determina su rol consultando sus coordenadas de hardware integradas.

Hola, Este es el hilo 0 del bloque 2
con un valor de $e=2.718280$

Hola, Este es el hilo 1 del bloque 2
con un valor de $e=2.718280$

Hola, Este es el hilo 2 del bloque 2
con un valor de $e=2.718280$

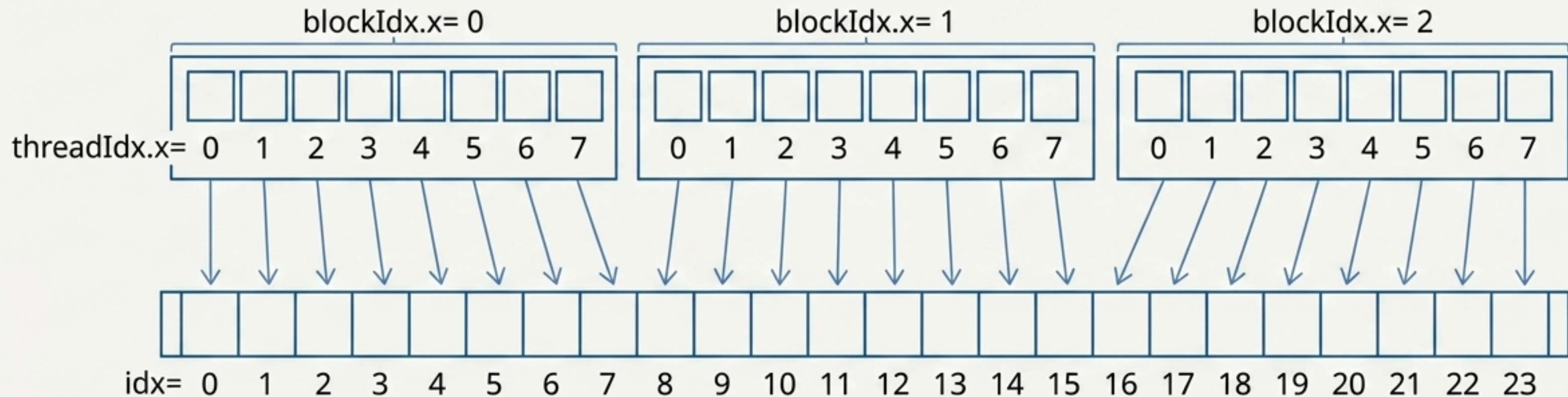
Hola, Este es el hilo 3 del bloque 2
con un valor de $e=2.718280$



Gestión de Índices: El Reto de la Memoria Lineal

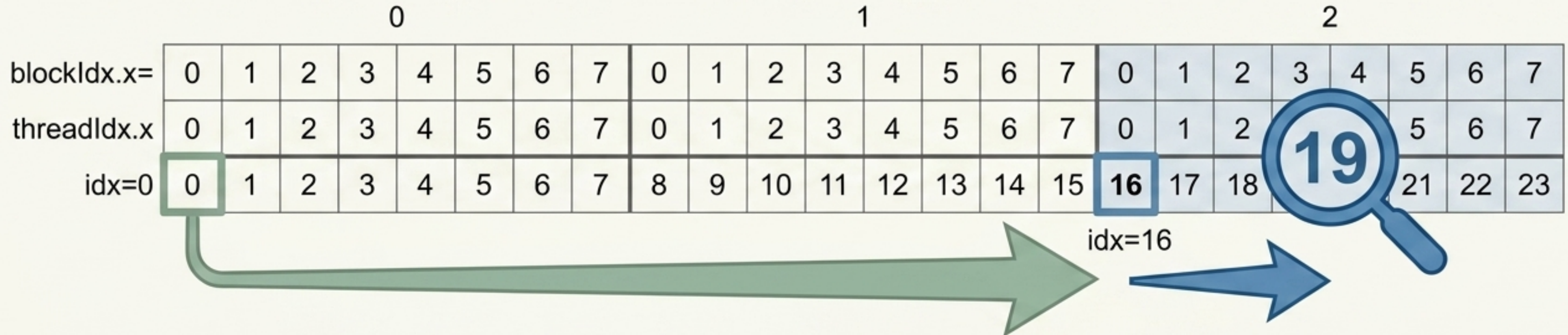
La memoria global de la GPU es una estructura 1D, pero los hilos se organizan en bloques encapsulados. Usamos esta fórmula para que cada hilo encuentre su posición única sin colisiones.

```
int idx = (blockIdx.x * blockDim.x) + threadIdx.x;
```



Anatomía de la Fórmula idx

Desglosando el cálculo para el Hilo 3 dentro del Bloque 2 (**N=24**, **blockDim.x=8**):



1. Salto de Bloque

Multiplicar el ID del bloque por su tamaño nos sitúa en el primer elemento de ese bloque.

$$2 * 8 = 16$$

2. Desplazamiento Local

Sumar el ID del hilo nos da la posición exacta.

$$16 + 3 = 19$$

El Ciclo de Vida de los Datos (Frontera Host vs Device)

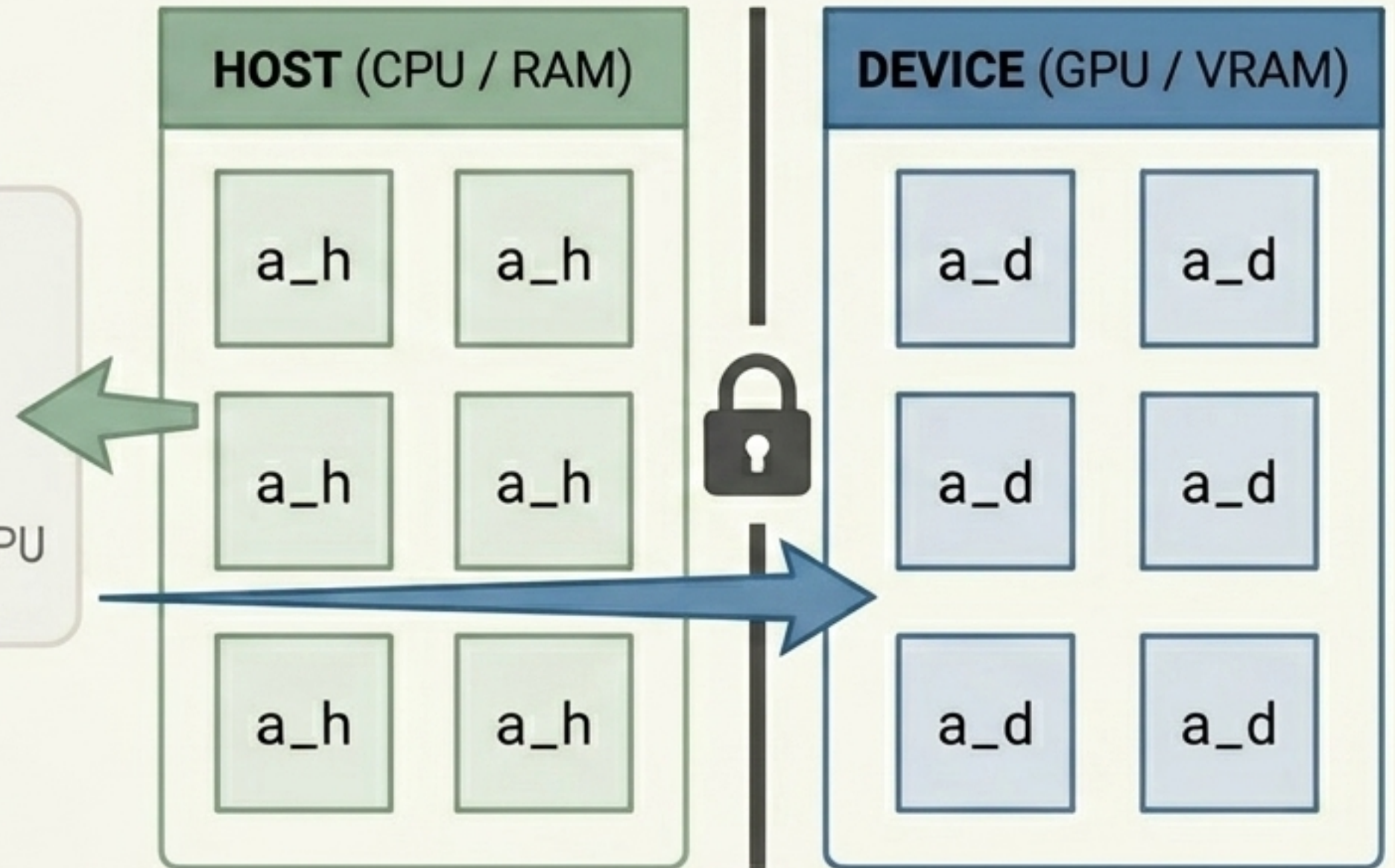
El CPU no puede acceder directamente a la memoria de la GPU y viceversa. Debemos gestionar dos ecosistemas de memoria paralelos y orquestar el movimiento entre ellos.

	HOST (CPU / RAM)	DEVICE (GPU / VRAM)
Nomenclatura	*_h (ej. a_h)	*_d (ej. a_d)
Asignación	malloc(size)	cudaMalloc((void*)&ptr, size)
Liberación	free(ptr)	cudaFree(ptr)

Operaciones Vectoriales: Preparación y Asignación

El primer paso es reservar espacio. `cudaMalloc` opera en la VRAM, requiriendo un doble puntero para modificar la dirección de memoria directamente desde el Host.

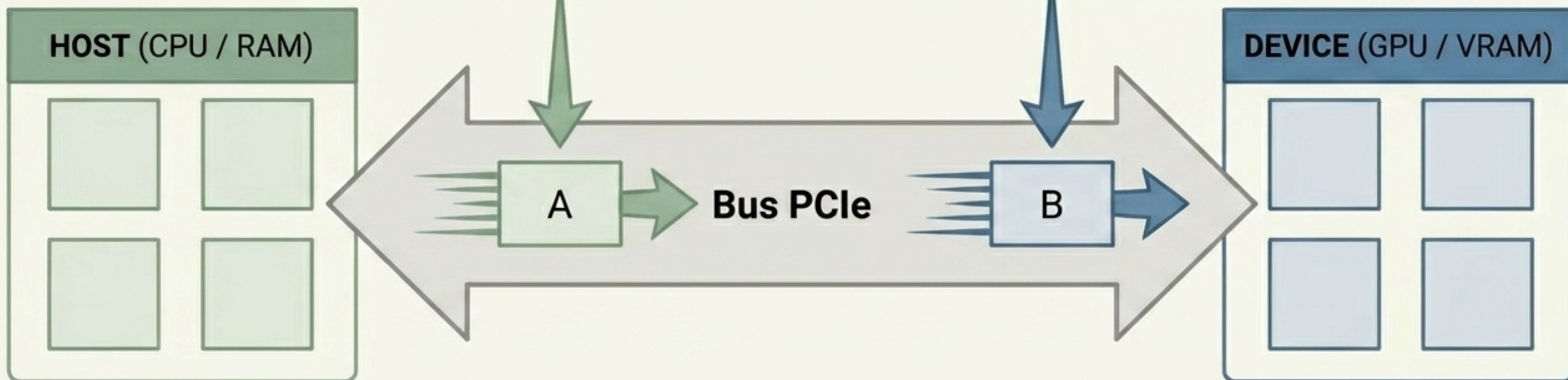
```
size_t size = N * sizeof(float);  
a_h = (float *)malloc(size); // CPU  
cudaMalloc((void **)&a_d, size); // GPU
```



Operaciones Vectoriales: Transferencia (H2D)

Los datos inicializados en el CPU deben cruzar físicamente el bus PCIe hacia la memoria global de la GPU antes del cálculo.

```
cudaMemcpy(a_d, a_h, size, cudaMemcpyHostToDevice);  
cudaMemcpy(b_d, b_h, size, cudaMemcpyHostToDevice);
```

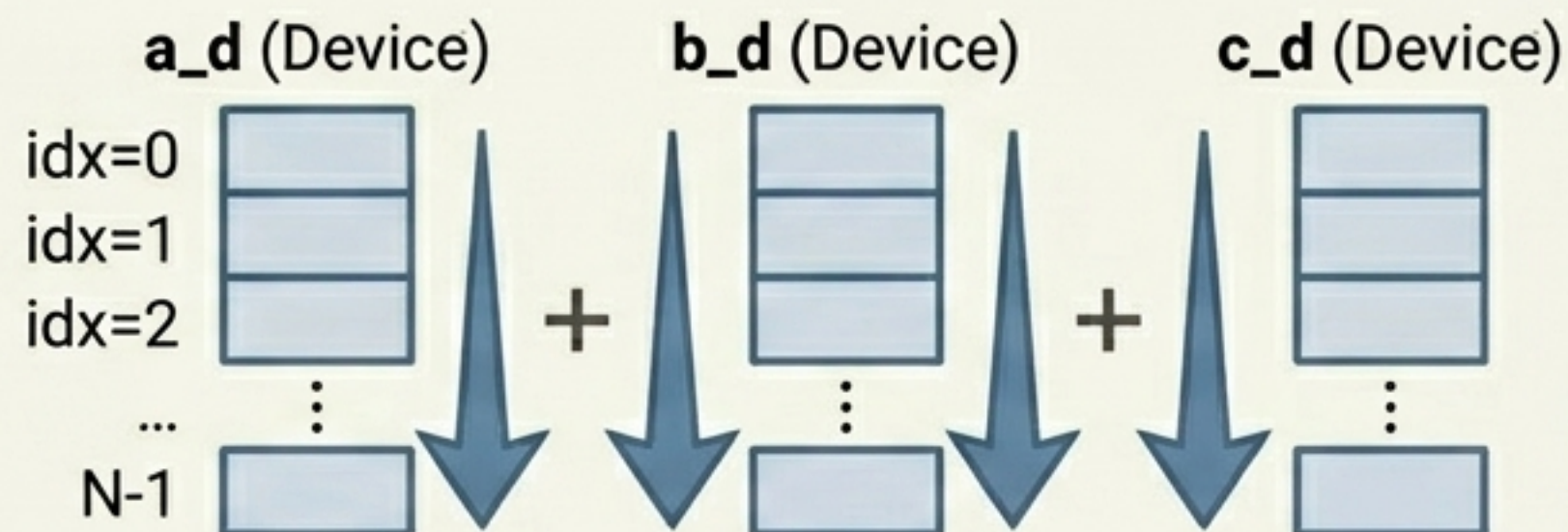


Operaciones Vectoriales: Ejecución y Retorno (D2H)

Cada hilo procesa un índice simultáneamente. El condicional `if(idx < N)` previene accesos fuera de límites. Finalmente, extraemos los resultados.

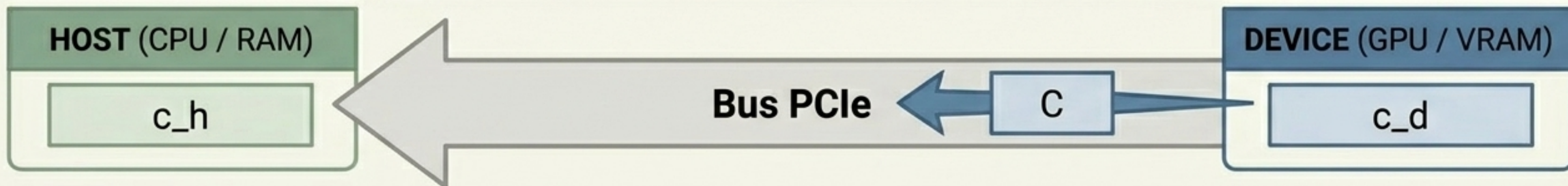
Ejecución

```
if (idx < N) {  
    c_d[idx] = a_d[idx] + b_d[idx];  
}
```



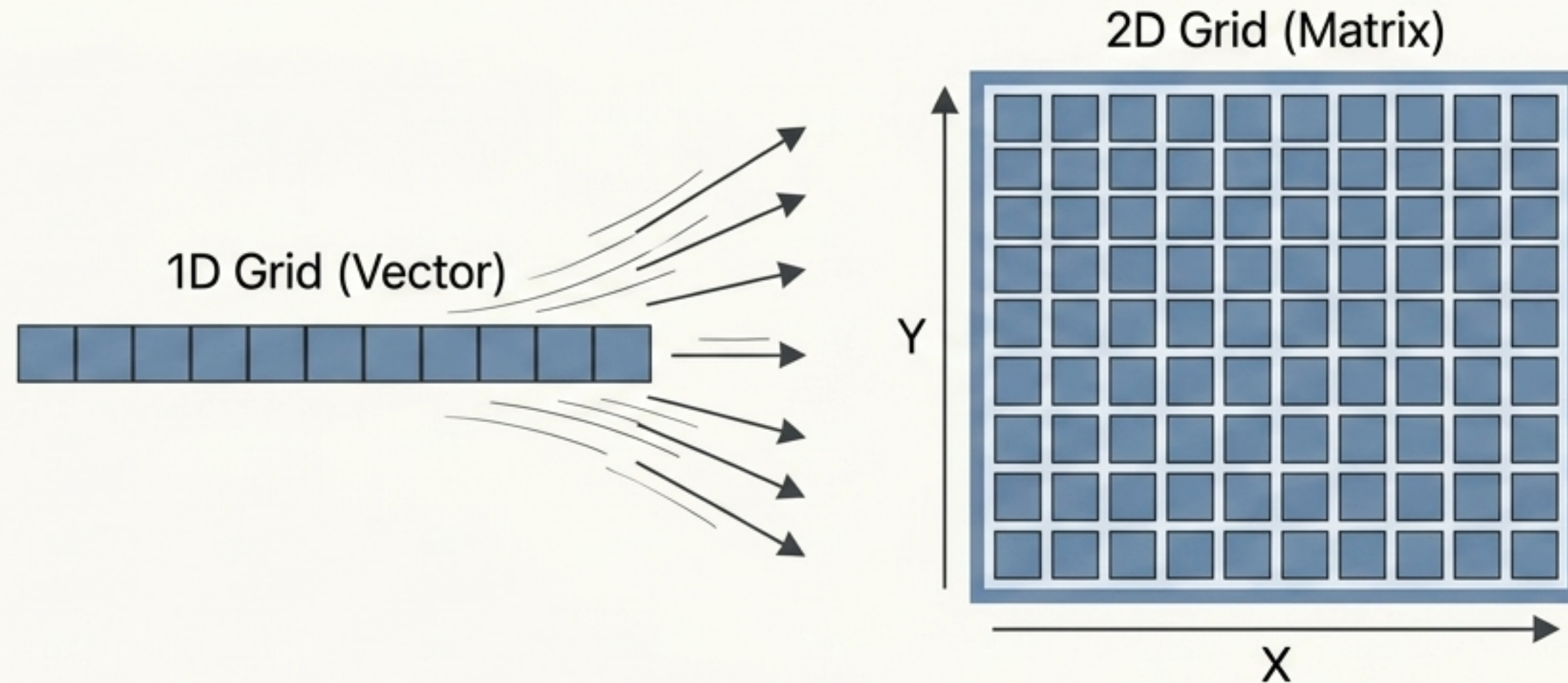
Retorno

```
cudaMemcpy(c_h, c_d, size, cudaMemcpyDeviceToHost);
```



Escalando la Complejidad: Hacia la Computación Matricial

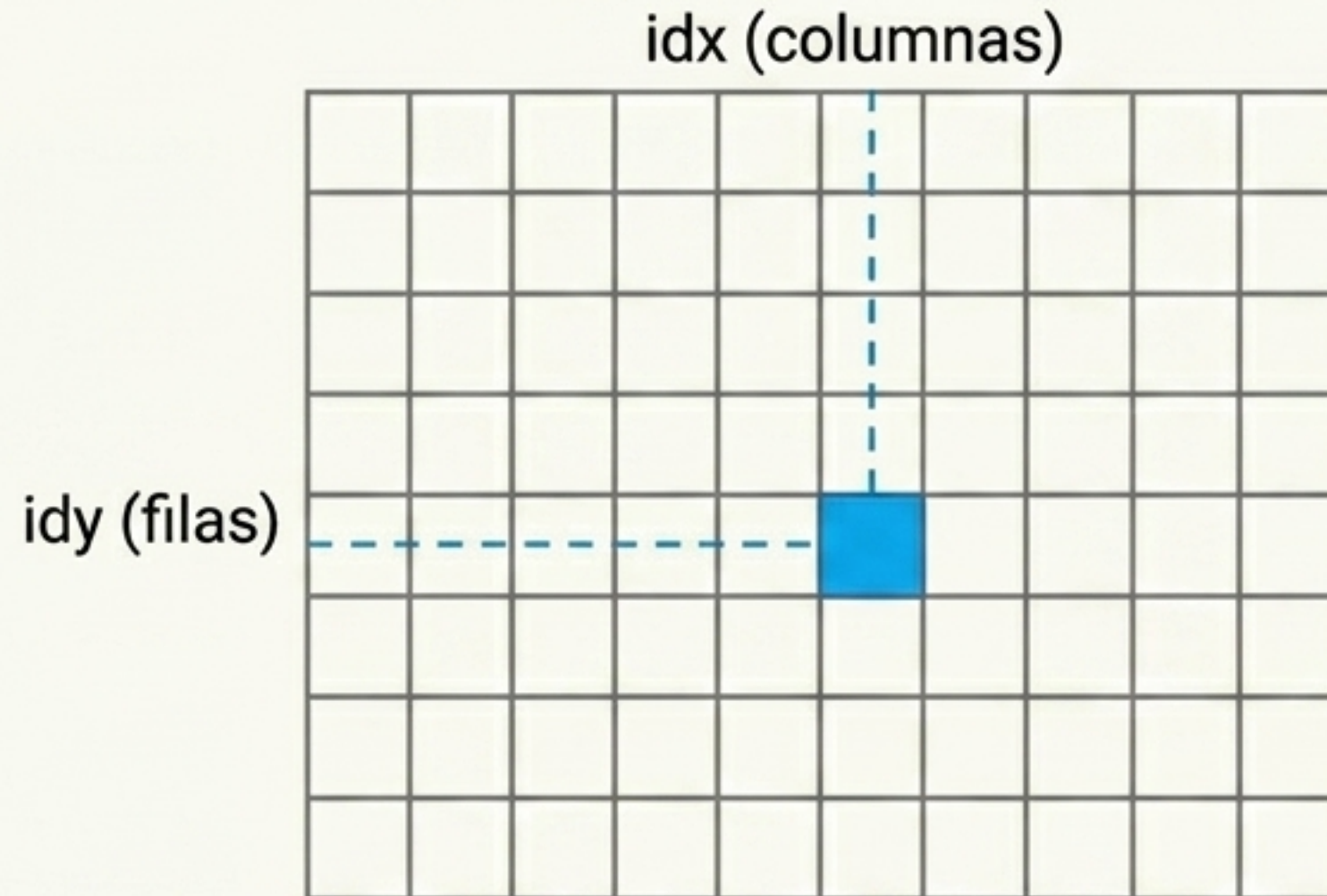
El procesamiento 1D es insuficiente para imágenes o álgebra lineal. CUDA escala elegantemente utilizando el tipo de dato dim3, permitiendo mallas en X, Y y Z.



```
// Definiendo una malla 2D
dim3 block_size(BLOCK_SIZE, BLOCK_SIZE);
dim3 n_blocks(div_up(ncol, block_size.x), div_up(nfil, block_size.y));
```

Coordenadas 2D: Mapeando Filas y Columnas

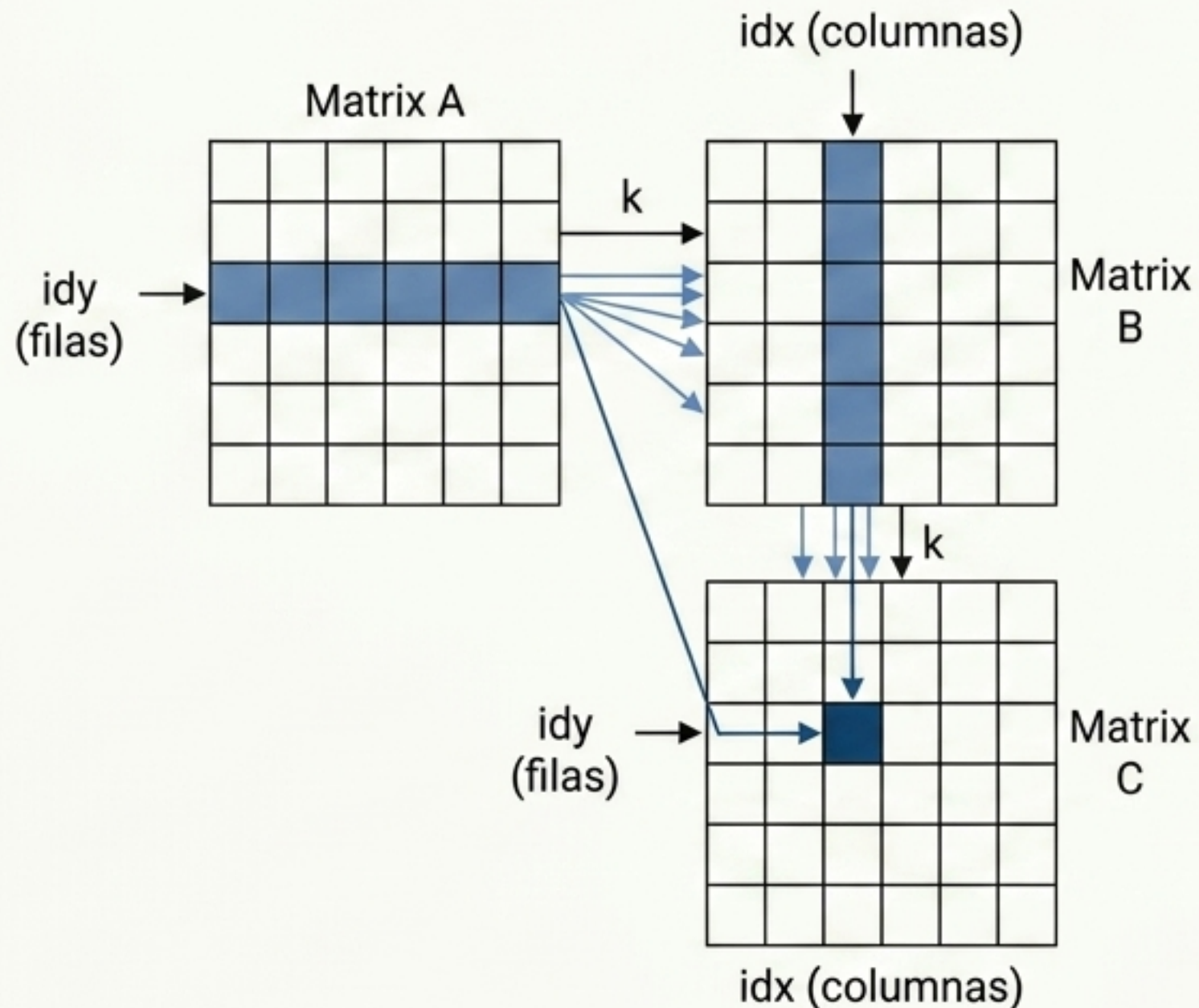
Replicamos la lógica de índices para ambos ejes. `idx` gobierna la posición horizontal (columnas), e `idy` gobierna la posición vertical (filas).



```
int idx = blockIdx.x * blockDim.x + threadIdx.x;  
int idy = blockIdx.y * blockDim.y + threadIdx.y;
```

El Bucle Interno: Multiplicación en Memoria Linealizada

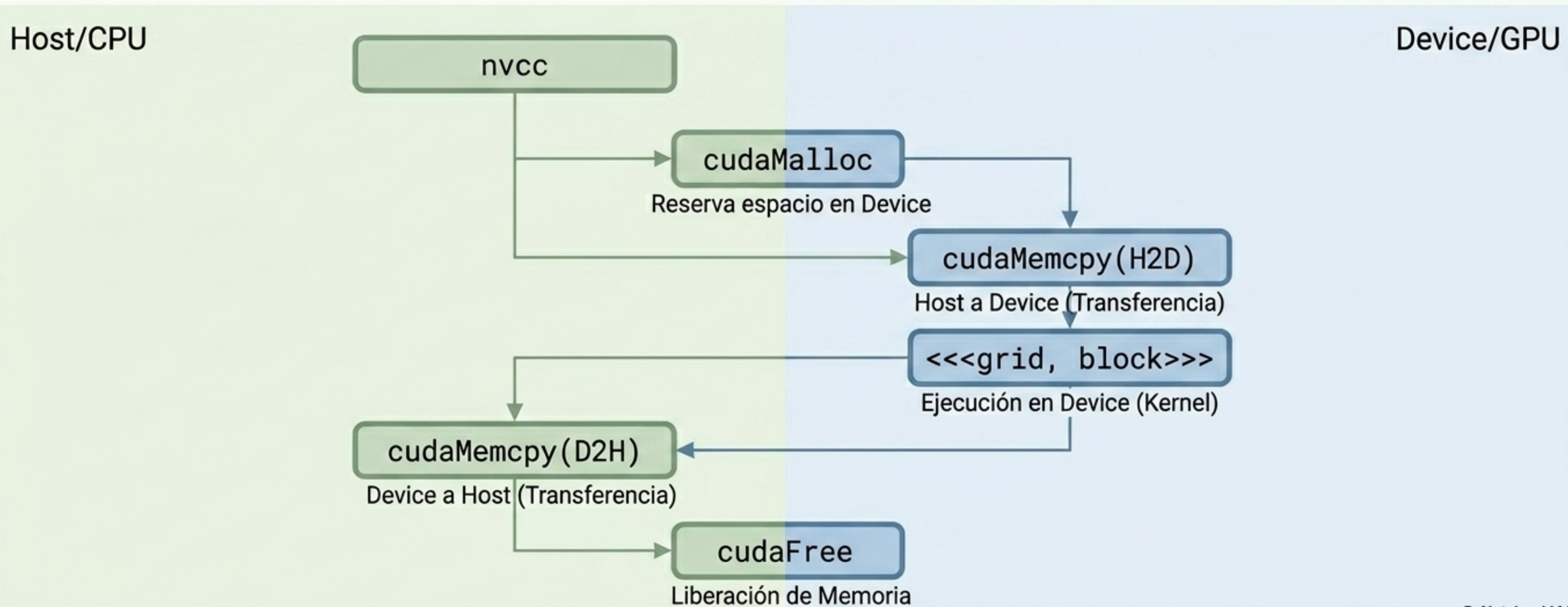
La memoria global sigue siendo físicamente plana. Aplanamos las coordenadas 2D y el hilo acumula el producto punto iterando sobre la fila y la columna.



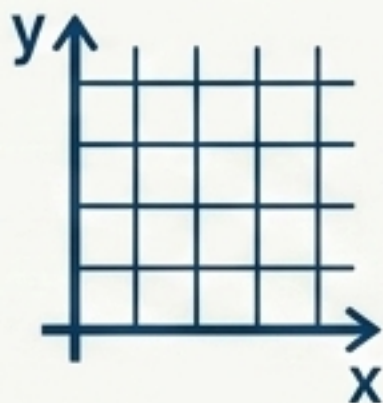
```
int index = idy * ncol + idx;
if (idy < nfil && idx < ncol) {
    float sum = 0.0f;
    for (int k = 0; k < ncol; k++) {
        sum += A[idy * ncol + k] * B[k * ncol + idx];
    }
    C[index] = sum;
}
```

Síntesis: El Ecosistema y Ciclo de la API de CUDA

El flujo de trabajo heterogéneo sigue un patrón estricto. La maestría radica en comprender y orquestar este ciclo.



Resumen Arquitectónico y Buenas Prácticas



Identidad Espacial

Siempre mapea explícitamente los hilos a los datos usando `idx` e `idy` para prevenir condiciones de carrera.



Límites de Transferencia

El cuello de botella es el Bus PCIe. Minimiza el uso de `cudaMemcpy`; mantén los datos en la GPU el mayor tiempo posible.



Gestión Estricta

No existe un recolector de basura nativo. Cada `cudaMalloc` debe cerrarse con un `cudaFree` para prevenir fugas de VRAM.

Comienza a programar.